

Practical Week 11

Session 11

Task 1

Develop a multithreaded counter application using POSIX threads (pthread_create, pthread_join). Demonstrate race conditions by allowing multiple threads to update a shared variable concurrently.



```
GNU nano 8.7.1 practical11_task1_race.c *
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>

#define NUM_THREADS 4
#define ITERATIONS 1000000

int counter = 0;

void *increment_counter(void *arg)
{
    (void)arg;

    for (int i = 0; i < ITERATIONS; i++)
    {
        counter++;
    }

    return NULL;
}

int main(void)
{
    pthread_t threads[NUM_THREADS];

    printf("=====\n");
    printf("Practical Session 11 - Task 1\n");
    printf("Multithreaded Counter - Race Condition\n");
    printf("=====\n");

    for (int i = 0; i < NUM_THREADS; i++)
    {
        if (pthread_create(&threads[i], NULL,
                          increment_counter, NULL) != 0)
        {
            perror("pthread_create");
            return 1;
        }
    }

    for (int i = 0; i < NUM_THREADS; i++)
    {
        if (pthread_join(threads[i], NULL) != 0)
        {
            perror("pthread_join");
        }
    }

    printf("Final Counter Value: %d\n", counter);
}
```

File to insert [from ./]:

^G Help	M-F New Buffer	^X Execute Command
^C Cancel	M-N No Conversion	^T Browse

```

    {
        perror("pthread_join");
        return 1;
    }
}

printf("\nExpected counter value: %d\n",
      NUM_THREADS * ITERATIONS);

printf("Actual counter value : %d\n", counter);

if (counter == NUM_THREADS * ITERATIONS)
{
    printf("Result: Correct\n");
}
else
{
    printf("Result: Race condition observed\n");
}

return 0;

```

Help	^O Write Out	^F Where Is	^K Cut	^T Execute	^C Location
Exit	^R Read File	^I Replace	^U Paste	^J Justify	^_ Go To Line

```

jella_pragathi@LAPTOP-RQQGT555:~$ nano practical11_task1_race.c
jella_pragathi@LAPTOP-RQQGT555:~$ gcc -Wall -Wextra -pthread practical11_task1_race.c -o practical11_task1_race
jella_pragathi@LAPTOP-RQQGT555:~$ ./practical11_task1_race
=====
Practical Session 11 - Task 1
Multithreaded Counter - Race Condition
=====

Expected counter value: 4000000
Actual counter value : 1276703
Result: Race condition observed
jella_pragathi@LAPTOP-RQQGT555:~$

```

Task 2

Modify the previous program using mutex locks (`pthread_mutex_lock`, `pthread_mutex_unlock`) and compare the outputs with the race-condition version. Analyze the impact of synchronization.



```
GNU nano 8.7.1 practical11_task2_mutex.c *
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>

#define NUM_THREADS 4
#define ITERATIONS 1000000

int counter = 0;

pthread_mutex_t counter_mutex = PTHREAD_MUTEX_INITIALIZER;

void *increment_counter(void *arg)
{
    (void)arg;

    for (int i = 0; i < ITERATIONS; i++)
    {
        pthread_mutex_lock(&counter_mutex);

        counter++;

        pthread_mutex_unlock(&counter_mutex);
    }

    return NULL;
}

int main(void)
{
    pthread_t threads[NUM_THREADS];

    printf("=====\n");
    printf("Practical Session 11 - Task 2\n");
    printf("Multithreaded Counter - Mutex\n");
    printf("=====\n");

    for (int i = 0; i < NUM_THREADS; i++)
    {
        if (pthread_create(&threads[i], NULL,
                          increment_counter, NULL) != 0)
        {
            perror("pthread_create");
            return 1;
        }
    }
}
```

Help **Write Out** **Where Is** **Cut** **Execute** **Location**
Exit **Read File** **Replace** **Paste** **Justify** **Go To Line**

```

    }

    for (int i = 0; i < NUM_THREADS; i++)
    {
        if (pthread_join(threads[i], NULL) != 0)
        {
            perror("pthread_join");
            return 1;
        }
    }

    printf("\nExpected counter value: %d\n",
        NUM_THREADS * ITERATIONS);

    printf("Actual counter value : %d\n", counter);

    if (counter == NUM_THREADS * ITERATIONS)
    {
        printf("Result: Correct - mutex synchronization successful\n");
    }
    else
    {
        printf("Result: Incorrect\n");
    }

    pthread_mutex_destroy(&counter_mutex);

    return 0;
}

```

^{^G} Help ^{^O} Write Out ^{^F} Where Is ^{^K} Cut ^{^T} Execute ^{^C} Location
^{^X} Exit ^{^R} Read File ^{^_\} Replace ^{^U} Paste ^{^J} Justify ^{^/} Go To Line

```

Result: Race Condition Observed
jella_pragathi@LAPTOP-RQQGT555:~$ nano practical11_task2_mutex.c
jella_pragathi@LAPTOP-RQQGT555:~$ gcc -Wall -Wextra -pthread practical11_task2_mutex.c -o p
ractical11_task2_mutex
jella_pragathi@LAPTOP-RQQGT555:~$ ./practical11_task2_mutex
=====
Practical Session 11 - Task 2
Multithreaded Counter - Mutex
=====

Expected counter value: 4000000
Actual counter value : 4000000
Result: Correct - mutex synchronization successful
jella_pragathi@LAPTOP-RQQGT555:~$

```